ALTKOM

Aplikacje kontenerowe w klastrach z wykorzystaniem Docker oraz Kubernetes

Dzień 1 · Docker i pierwsze kroki w Kubernetesie

Dawid Marniok

9–10 listopada 2026 · Altkom Akademia

Kim jestem



Dawid Marniok

DevOps Lead · architekt · trener

- Od lat w DevOps: kontenery, Kubernetes, CI/CD i chmura (Azure, AWS)
- Projektuję i wdrażam platformy oparte o Kubernetes w projektach produkcyjnych
- Zajmuję się też observability i bezpieczeństwem w pipeline'ach
- Szkolę z Dockera, Kubernetesa i narzędzi DevOps — praktycznie, na żywych przykładach

linkedin.com/in/dawid-marniok

Jak pracujemy przez dwa dni

Rytm tematu

- krótka teoria: po co i jak działa
- demo na moim ekranie
- lab: robicie sami
- omówienie i pytania

Laby

- każdy lab ma część podstawową i „dla chętnych”
- utknąłeś? pytaj od razu, nie czekaj
- skończyłeś wcześniej? bierz zadania z gwiazdką

Repozytorium

- laby, aplikacja ToDos, docs/ (ściągi i diagramy)
- setup/ — instalacja i klaster
- zostaje z Wami po szkoleniu

Agenda: jedna aplikacja od kontenera do klastra

Dzień 1 · Docker

- kontenery i podstawy
- sieci, dane i logi
- obrazy i Dockerfile
- Docker Compose: ToDos (web + api + baza)
- architektura Kubernetesa, własny klaster kind

Dzień 2 · Kubernetes

- Pod i Namespace
- Deployment, ConfigMap, Secret
- Service i Ingress
- ćwiczenie końcowe: ToDos na klastrze
- co dalej

Na start: sprawdź środowisko

Zanim przejdziemy do teorii

```
git clone <adres z czatu>
cd docker-k8s-altkom

./setup/sprawdz.sh
# ✅ Wszystko gotowe

./setup/sprawdz.sh --pobierz   # obrazy do labów
```

- Windows: Git Bash w Windows Terminal
- ❌ przy Dockerze → uruchom Docker Desktop
- ❌ przy porcie 80 → zamknij program, który go zajmuje
- Problem? Napisz od razu — rozwiążemy w przerwie

Wynik ✅ = gotowe na cały dzień 1. Klaster Kubernetes postawimy w temacie 5.

1

DZIEŃ 1 · TEMAT 1

Kontenery i podstawy

Po tym temacie uruchamiasz kontener, wiesz, co dzieje się pod spodem, i konfigurujesz go z zewnątrz.

1
Kontenery i podstawy

2
Sieci i dane

3
Obrazy i Dockerfile

4
Docker Compose

5
Kubernetes: architektura

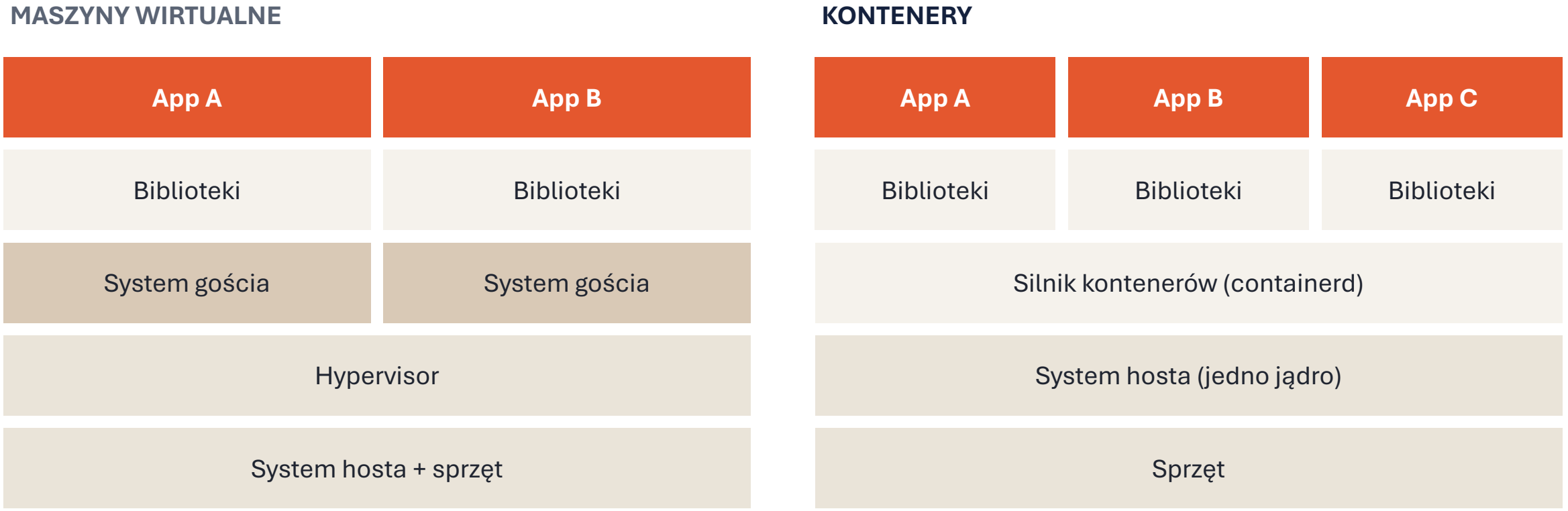
„U mnie działa” — problem, który rozwiązują kontenery

Laptop programisty	Serwer testowy	Produkcja
• Node 22, biblioteki w wersji z dziś • konfiguracja w pliku lokalnym • „działa”	• Node 18, inna wersja OpenSSL • brakuje jednej biblioteki systemowej • „nie działa, ale u mnie działa”	• jeszcze inny system • ręczna instalacja według dokumentu • każda zmiana = ryzyko

Kontener = aplikacja + wszystko, czego potrzebuje, zapakowane w jeden przenośny obraz

Maszyna wirtualna kontra kontener

Kontener nie ma własnego systemu operacyjnego — dzieli jądro z hostem



VM: minuty startu, gigabajty · kontener: sekundy startu, megabajty

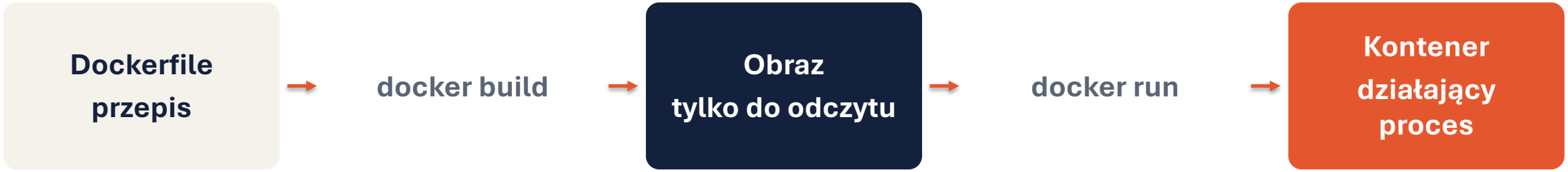
Kontener to zwykły proces z izolacją

- namespaces — proces widzi tylko swoje: procesy, sieć, pliki, nazwę hosta
- cgroups — limity: ile CPU i pamięci może zużyć
- system plików z obrazu + cienka warstwa do zapisu
- z punktu widzenia hosta: kolejny proces na liście ps
- zatrzymasz główny proces = zatrzymasz kontener

Główny proces kończy się → kontener się zatrzymuje

Obraz i kontener: przepis i danie

Z jednego obrazu uruchomisz dowolnie wiele kontenerów



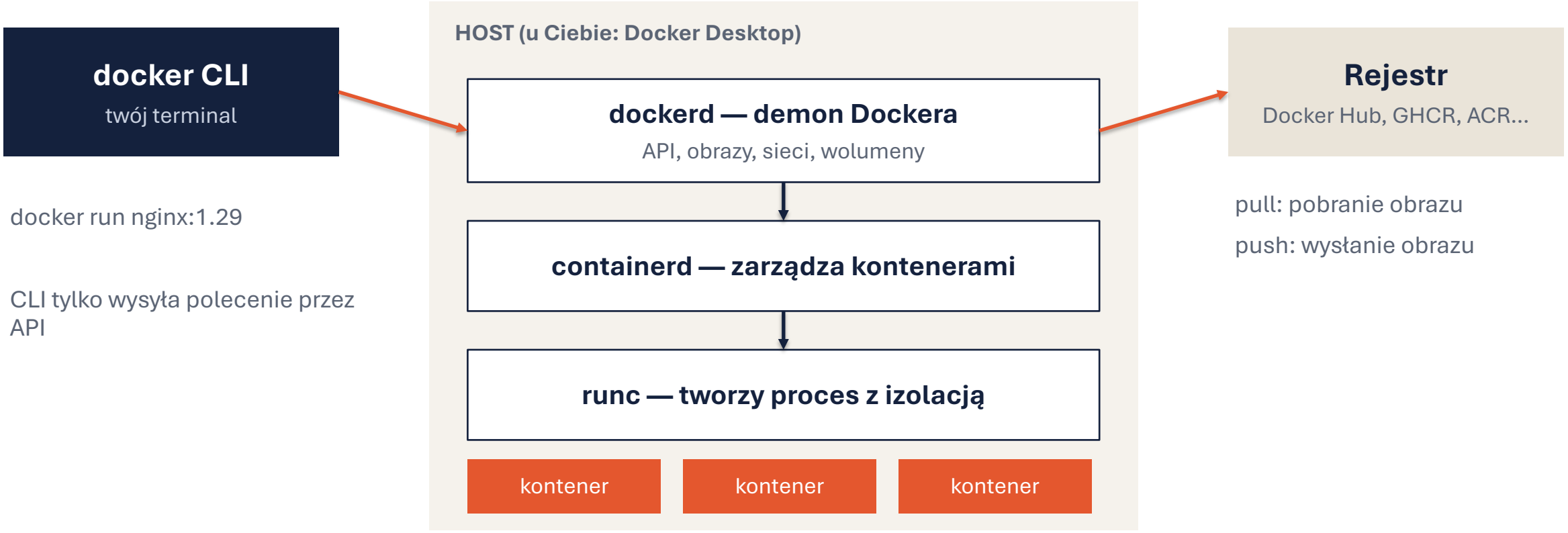
Obraz: warstwy plików + metadane (co uruchomić, jakie porty). Nie zmienia się.

Kontener: obraz + warstwa do zapisu + uruchomiony proces. Usunięty — znika z nim warstwa zapisu.

Rejestr (np. Docker Hub): sklep z obrazami — docker pull / docker push.

Obraz = klasa · kontener = obiekt (instancja)

Architektura Dockera: klient, demon, runtime, rejestr



docker run nginx:1.29

CLI tylko wysyła polecenie przez API

pull: pobranie obrazu

push: wysłanie obrazu

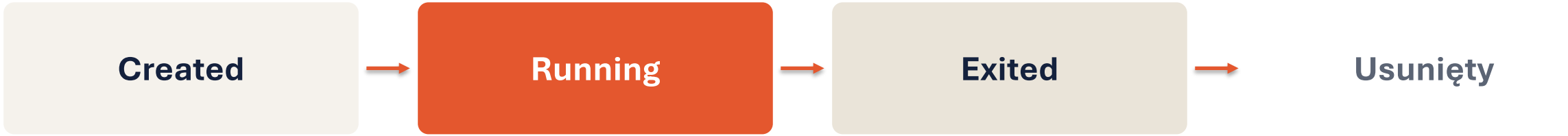
Kubernetes używa containerd i runc bezpośrednio — bez dockerd. Obrazy z Dockera działają tam bez zmian.

Polecenia na pierwszy dzień

Polecenie	Co robi
docker run -d --name web -p 8080:80 nginx:1.29	uruchamia w tle, port 8080 laptopa → 80 kontenera
docker ps · docker ps -a	działające · wszystkie kontenery
docker logs -f web	logi na żywo
docker exec -it web sh	powłoka w działającym kontenerze
docker stop web · docker start web	zatrzymaj · uruchom ponownie
docker rm -f web	zatrzymaj i usuń
docker run -e KLUCZ=wartosc ...	konfiguracja przez zmienną środowiskową
docker run -v dane:/sciezka ...	wolumen: dane przeżywają kontener

Cykl życia kontenera

Zatrzymany kontener nadal istnieje — razem z warstwą zapisu i konfiguracją



docker create (rzadko używane)	docker run / start proces działa	docker stop proces zakończony, kontener zostaje	docker rm warstwa zapisu znika

```
docker ps -a --format 'table {{.Names}}\t{{.Status}}'
web     Exited (0) 2 minutes ago        ← zatrzymany, ale istnieje: docker start web
```

docker stop ≠ docker rm. Zatrzymane kontenery zajmują miejsce — sprzątaj je.

Pułapka: kontener jest ulotny

Co się dzieje

- aplikacja zapisuje dane w kontenerze
- docker rm albo nowa wersja obrazu
- warstwa zapisu znika — dane też

Jak to robić dobrze

- dane trzymaj w wolumenie: -v dane:/sciezka
- konfigurację podawaj z zewnątrz: -e
- kontener ma dać się usunąć i odtworzyć w każdej chwili

Kontener ma być do wyrzucenia: dane w wolumenie, konfiguracja w zmiennych

DEMO

Pierwszy kontener

Uruchamiam, zaglądam do środka, zatrzymuję — i sprawdzam, co zostało

```
docker run --rm -it busybox:1.37 sh
  ps ; hostname ; uname -r ; exit

docker run -d --name apache -p 8090:80 httpd:2.4
docker ps ; curl localhost:8090
docker logs apache
docker exec -it apache bash   # zmieniam index.html

docker stop apache ; docker ps -a
docker start apache ; curl localhost:8090
docker rm -f apache          # i od nowa: zmiana znika
```

OBSERWUJCIE

- sh ma PID 1 i widzi tylko kilka procesów
- uname -r: jądro hosta — kontener nie ma własnego
- logi Apache pokazują żądanie z curla
- zmiana przeżywa stop, ale nie rm

LAB `labs/lab01-docker-podstawy/`

Pierwsze kontenery

1. kontener interaktywny i serwer WWW w tle
2. logi, exec, stop, start, rm — cały cykl życia
3. konflikt portów i zmienne środowiskowe (postgres)
4. wolumen: dane, które przeżywają kontener

SUKCES: nginx pod localhost:8080 i plik w wolumenie po usunięciu kontenerów

2

DZIEŃ 1 · TEMAT 2

Sieci i dane

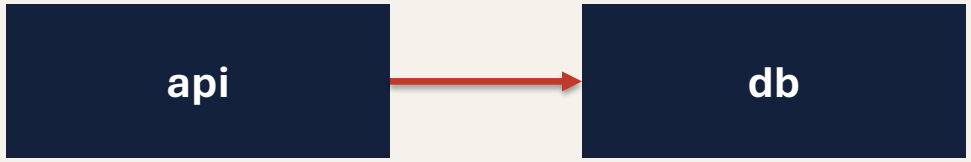
Po tym temacie łączysz kontenery po nazwie i wiesz, gdzie trzymać dane i logi.

1	2	3	4	5
Kontenery i podstawy	**Sieci i dane**	Obrazy i Dockerfile	Docker Compose	Kubernetes: architektura

Własna sieć = DNS po nazwie kontenera

Domyślna sieć bridge nie zna nazw. Sieć utworzona przez Ciebie — zna.

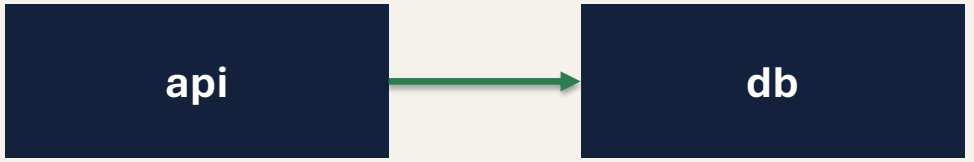
sieć domyślna „bridge”



http://db:5432 ✗ nieznana nazwa

(tylko adresy IP, które się zmieniają)

`docker network create app-net`



http://db:5432 ✓ działa — Docker zamienia nazwę na IP

```
docker network create app-net
docker run -d --name db  --network app-net -e POSTGRES_PASSWORD=changeme postgres:17
docker run -d --name api --network app-net -e DB_HOST=db api:1.0
```

Adresy IP kontenerów się zmieniają — łącz się zawsze po nazwie

Gdzie trzymać dane: trzy sposoby

Sposób	Przykład	Kiedy
Wolumen	-v pgdata:/var/lib/postgresql/data	dane aplikacji: bazy, pliki użytkowników
Bind mount	-v "$(pwd)/strona:/usr/share/nginx/html:ro"	kod lub konfiguracja z laptopa w trakcie pracy
tmpfs	--tmpfs /tmp	pliki tymczasowe, tylko w pamięci
nic	(warstwa zapisu kontenera)	nic nie musi przetrwać — znika z kontenerem

Dane w wolumenie, kod w obrazie, konfiguracja w zmiennych

Logi: aplikacja pisze na stdout, nie do pliku

- Docker zbiera wszystko, co proces wypisze na stdout i stderr
- docker logs web · -f na żywo · --tail 50 · --since 10m
- pliki logów w kontenerze znikają razem z nim i rosną bez kontroli
- limit rozmiaru: --log-opt max-size=10m --log-opt max-file=3
- Kubernetes działa tak samo: kubectl logs czyta stdout poda

DEMO

Sieć, wolumen i logi

Baza w kontenerze: łączę się po nazwie, usuwam kontener — dane zostają

```
docker network create demo
docker run -d --name baza --network demo \
  -v demo-dane:/data/db mongo:7.0
docker run --rm --network demo busybox:1.37 nslookup baza
docker network inspect demo        # IP kontenera

docker exec baza mongosh --quiet --eval \
  'db.notatki.insertOne({t: "przetrwam"})'
docker stop baza ; docker rm baza
docker run -d --name baza --network demo \
  -v demo-dane:/data/db mongo:7.0
docker exec baza mongosh --quiet --eval 'db.notatki.find()'
docker logs --tail 5 baza
```

OBSERWUJCIE

- nslookup: nazwa baza zamienia się na IP z sieci demo
- kontener usunięty, a notatka jest — siedzi w wolumenie
- nowy kontener może dostać inny IP; nazwa zostaje
- logi bazy czytamy z stdout, bez wchodzenia do środka

LAB `labs/lab02-sieci-wolumeny/`

Sieci, dane i logi

1. domyślna sieć kontra własna: DNS po nazwie
2. izolacja: kontenery z innej sieci się nie widzą
3. bind mount: strona z laptopa w nginx, edycja na żywo
4. wolumen bazy: tabela przeżywa usunięcie kontenera

SUKCES: SELECT pokazuje dane po usunięciu i odtworzeniu kontenera postgres

3

DZIEŃ 1 · TEMAT 3

Obrazy i Dockerfile

Po tym temacie budujesz własny obraz: szybko (cache), mały (multi-stage) i bezpieczny (bez roota).

1 Kontenery i podstawy

2 Sieci i dane

3 Obrazy i Dockerfile

4 Docker Compose

5 Kubernetes: architektura

Obraz to warstwy — każda instrukcja dokłada jedną

`CMD ["node", "server.js"]`	metadane, 0 B
`COPY server.js ./`	kod: 4 KB
`RUN npm ci --omit=dev`	zależności: 12 MB
`COPY package*.json ./`	2 KB
`FROM node:22-alpine`	system + Node: 160 MB

- warstwy są tylko do odczytu i współdzielone między obrazami
- zmiana w jednej warstwie = przebudowa jej i wszystkich nad nią
- niezmienione warstwy idą z cache

To, co zmienia się rzadko — na dół. To, co zmienia się często (kod) — na górę.

Dockerfile: instrukcje, których używasz na co dzień

Instrukcja	Co robi
FROM node:22-alpine	obraz bazowy — zawsze z konkretną wersją
WORKDIR /app	katalog roboczy (tworzy go, jeśli nie ma)
COPY package*.json ./	kopiuje pliki z kontekstu budowania do obrazu
RUN npm ci --omit=dev	wykonuje polecenie podczas budowania → nowa warstwa
ENV PORT=3000	zmienna środowiskowa (można nadpisać przy docker run -e)
EXPOSE 3000	dokumentacja: na którym porcie słucha aplikacja
USER node	kolejne polecenia i aplikacja działają jako ten użytkownik
CMD ["node", "server.js"]	co uruchomić przy starcie kontenera

Kolejność instrukcji decyduje o cache

ŹLE: każda zmiana kodu = npm install od nowa

```
FROM node:22-alpine
WORKDIR /app
COPY . .
RUN npm ci
CMD ["node", "server.js"]
```

zmiana jednej linii w server.js → 60 s budowania

DOBRZE: zależności z cache

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
CMD ["node", "server.js"]
```

zmiana w server.js → 2 s, npm ci: CACHED

Plus .dockerignore z node_modules i .git — mniejszy kontekst, szybszy build

Multi-stage: buduj w jednym obrazie, uruchamiaj w drugim

```
# etap 1: kompilacja — cały toolchain
FROM golang:1.25 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /hello .

# etap 2: tylko gotowy program
FROM gcr.io/distroless/static-debian12
COPY --from=build /hello /hello
USER nonroot
CMD ["/hello"]
```

jeden etap

953 MB

multi-stage

9,6 MB

zmierzone na programie z lab03: ten sam kod, 100× mniejszy obraz

Mniejszy obraz = szybszy pull, mniej podatności, brak kompilatora na produkcji

Dobre nawyki od pierwszego obrazu

Bezpieczeństwo

- USER: aplikacja nie działa jako root
- mały obraz bazowy (alpine, slim, distroless)
- sekrety nie w Dockerfile ani ENV obrazu
- skanuj obrazy: docker scout, trivy

Przewidywalność

- konkretne tagi: node:22-alpine, nie :latest
- npm ci zamiast npm install
- jedna aplikacja = jeden kontener
- konfiguracja przez zmienne, nie w obrazie

Szybkość

- kolejność: zależności przed kodem
- .dockerignore
- multi-stage dla kompilowanych
- łącz RUN apt-get update && install

Rejestr: skąd Kubernetes weźmie Twój obraz

Nazwa obrazu mówi, gdzie on leży

```
ghcr.io/moja-firma/todos-api:1.0
```

rejestr (domyślnie docker.io)	organizacja lub użytkownik	repozytorium	tag = wersja

Docker Hub	publiczne obrazy, limity pobrań bez logowania
GHCR · GitLab Registry	obok kodu, uprawnienia jak do repo
ACR · ECR · Artifact Registry	w chmurze, blisko klastra
Harbor	własny rejestr on-premises, skanowanie

DEMO

Build, cache i multi-stage

Ten sam kod, różna kolejność instrukcji i różne rozmiary obrazów

```
cd demo/d1-03-build-cache-multistage/python
docker build -t demo-py:1.0 .
# zmiana w app.py
docker build -t demo-py:1.0 .      # CACHED?
docker build -f Dockerfile.zla-kolejnosc \
  -t demo-py:zla .

cd ../c
docker build -f Dockerfile.jeden-etap -t demo-c:duzy .
docker build -t demo-c:maly .
docker image ls demo-c
```

OBSERWUJCIE

- które kroki mają CACHED po zmianie kodu
- ile trwa drugi build w porównaniu z pierwszym
- rozmiar: 244 MB kontra 136 kB

LAB `labs/lab03-dockerfile/`

Własny obraz z Dockerfile

1. nginx: strona wbudowana w obraz (FROM + COPY)
2. Node.js: cache warstw, .dockerignore, USER node
3. Go: obraz jednoetapowy kontra multi-stage
4. tagi i historia warstw

SUKCES: curl localhost:3000 odpowiada „użytkownik: node”, a hello-go:maly ma ok. 10 MB

4

DZIEŃ 1 · TEMAT 4

Docker Compose

Po tym temacie uruchamiasz aplikację z kilku kontenerów jednym poleceniem — i rozumiesz, gdzie Compose się kończy.

1 Kontenery i podstawy

2 Sieci i dane

3 Obrazy i Dockerfile

4 Docker Compose

5 Kubernetes: architektura

Trzy kontenery z ręki = siedem poleceń

Kolejność trzeba pamiętać, literówka psuje całość

```
docker network create todos
docker volume create mongo-data
docker run -d --name mongo --network todos -v mongo-data:/data/db mongo:7.0
docker build -t todos-api:1.0 app/todos/api
docker run -d --name api --network todos -e MONGO_URL=mongodb://mongo:27017 todos-api:1.0
docker build -t todos-web:1.0 app/todos/web
docker run -d --name web --network todos -p 8080:8080 todos-web:1.0
```

- kto pamięta kolejność? kto zaktualizuje jedną usługę bez psucia reszty?
- jak przekazać to koledze z zespołu — w README?

Docker Compose: ta sama konfiguracja w jednym pliku, opisana deklaratywnie

compose.yaml: opisujesz co ma działać, a nie jak

```yaml
name: todos
services:
  mongo:
    image: mongo:7.0
    volumes:
      - mongo-data:/data/db

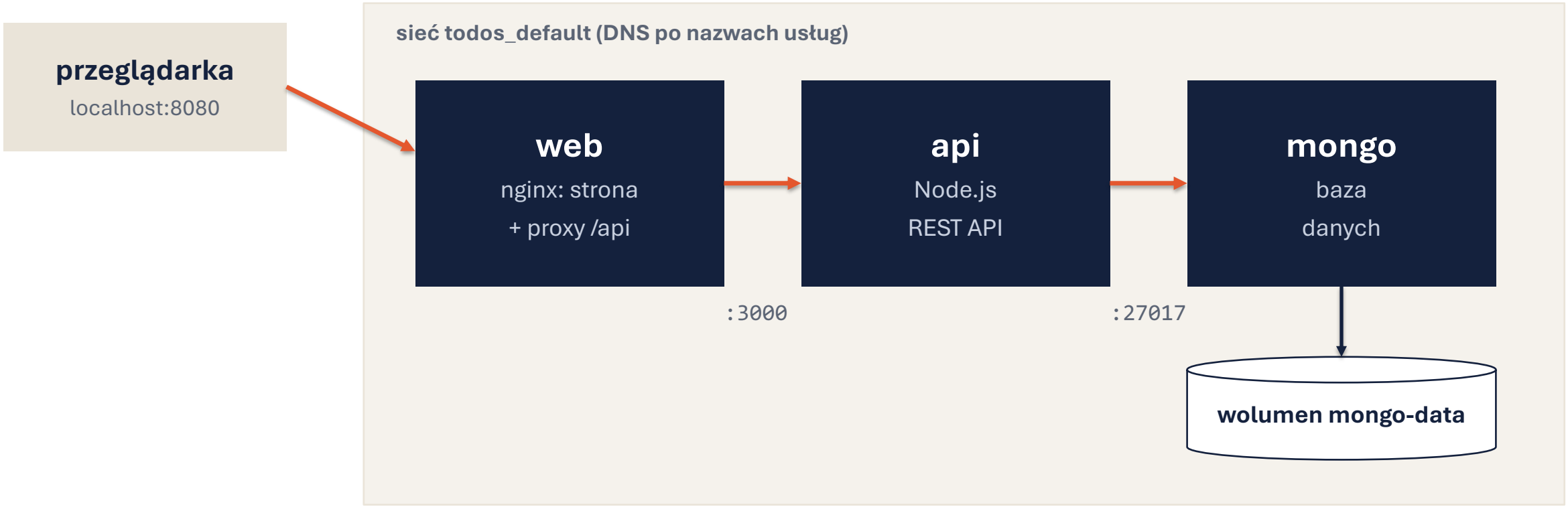
  api:
    build: ./api
    environment:
      MONGO_URL: mongodb://mongo:27017
    depends_on: [mongo]

  web:
    build: ./web
    ports:
      - "8080:8080"

volumes:
  mongo-data:
```

- services — każdy kontener to usługa
- nazwa usługi = nazwa w DNS (mongo, api)
- sieć tworzy się sama
- build albo image — budujesz lub pobierasz
- bez pola version: — od Compose v2 zbędne

ToDos w Compose: trzy usługi, jedna sieć, jeden wolumen



Tylko web ma port na zewnątrz. api i mongo są osiągalne wyłącznie wewnątrz sieci.

Kolejność startu: depends_on + healthcheck

```
services:
  mongo:
    image: mongo:7.0
    healthcheck:
      test: ["CMD", "mongosh", "--quiet",
             "--eval", "db.adminCommand('ping').ok"]
      interval: 5s
      retries: 10

  api:
    depends_on:
      mongo:
        condition: service_healthy
```

- samo depends_on czeka tylko na start kontenera
- baza startuje kilka sekund dłużej niż jej kontener
- service_healthy czeka, aż healthcheck przejdzie
- i tak pisz aplikację tak, żeby ponawiała połączenie

docker compose: polecenia na co dzień

Polecenie	Co robi
docker compose up -d --build	buduje obrazy i uruchamia wszystko w tle
docker compose ps	stan usług
docker compose logs -f api	logi jednej usługi na żywo
docker compose exec mongo mongosh	polecenie w kontenerze usługi
docker compose up -d --scale api=3	trzy instancje api
docker compose down	usuwa kontenery i sieć, wolumeny zostają
docker compose down -v	usuwa także wolumeny — dane znikają

Compose jest świetny — do granic jednego hosta

Compose wystarcza

- środowisko programisty
- testy integracyjne w CI
- małe aplikacje na jednym serwerze
- demo, prototyp

Compose nie wystarcza

- padł serwer = padła aplikacja
- skalowanie na wiele maszyn
- aktualizacja bez przerwy dla użytkowników
- setki usług wielu zespołów

Jeden host i ręczne naprawy → Compose. Wiele maszyn i samonaprawa → Kubernetes.

DEMO

Docker Compose

Dwie usługi z jednego pliku: start, DNS po nazwie usługi, dane po down

```
# demo/d1-04-compose/compose.yaml
services:
  web:
    image: nginx:1.29
    ports: ["8080:80"]
  db:
    image: postgres:17
    environment: {POSTGRES_PASSWORD: demo}
    volumes: [dane:/var/lib/postgresql/data]
volumes:
  dane:

docker compose up -d ; docker compose ps
docker compose exec web getent hosts db
docker compose logs db ; docker compose down
```

OBSERWUJCIE

- jedno polecenie tworzy sieć, wolumen i dwa kontenery
- web widzi db po nazwie usługi — sieć Compose ma DNS
- nazwy: projekt-usługa-numer, np. d1-04-compose-db-1
- down usuwa kontenery i sieć, wolumen zostaje

LAB

`labs/lab04-compose-todos/`

ToDos w Docker Compose

1. uruchom wersję startową i dopisz usługę web
2. wolumen: zadania przeżywają docker compose down
3. healthcheck + depends_on: API startuje po bazie
4. --scale api=3: obserwuj nazwę hosta na stronie

SUKCES: http://localhost:8080 działa, a zadania zostają po down i up

5

DZIEŃ 1 · TEMAT 5

Kubernetes: architektura

Po tym temacie wiesz, z czego składa się klaster, jak myśli Kubernetes i masz własny klaster na laptopie.

1 Kontenery i podstawy

2 Sieci i dane

3 Obrazy i Dockerfile

4 Docker Compose

5 Kubernetes: architektura

Problem skali: setki kontenerów na dziesiątkach maszyn

- gdzie uruchomić kontener, żeby starczyło CPU i pamięci?
- kto podniesie kontenery, gdy w nocy padnie serwer?
- jak wdrożyć nową wersję bez przerwy dla użytkowników?
- jak znaleźć usługę, której adres ciągle się zmienia?
- jak dodać instancje w szczycie i zabrać je w nocy?

Orkiestrator odpowiada na te pytania automatycznie — 24 godziny na dobę

Kubernetes: standardowy orkiestrator kontenerów

Samonaprawa

pod padł → nowy; węzeł padł → pody na innym węźle

Skalowanie

ręcznie jednym poleceniem albo automatycznie wg obciążenia

Wdrożenia

rolling update bez przerwy i rollback jednym poleceniem

Odkrywanie usług

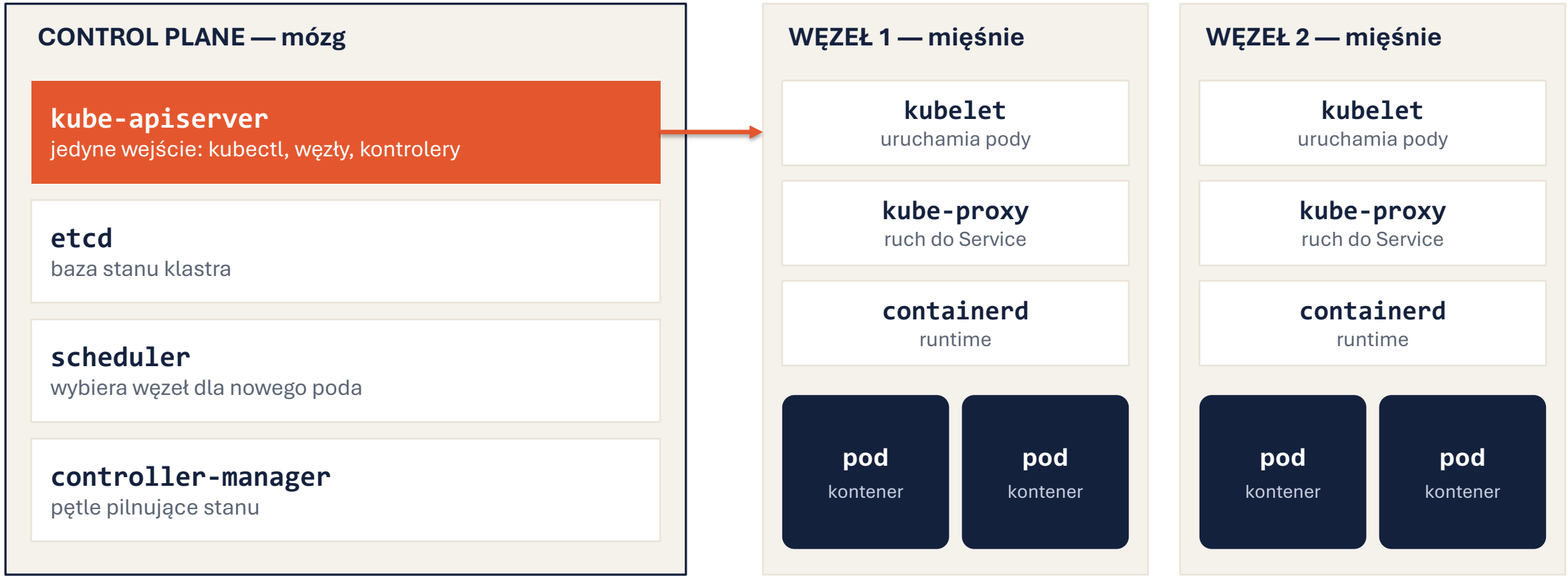
stała nazwa i adres dla zmieniających się podów

Google (2014), na bazie wewnętrznego systemu Borg · rozwijany przez CNCF · napisany w Go

dostępny wszędzie: AKS, EKS, GKE, OpenShift, Rancher, on-premises, laptop (kind, minikube)

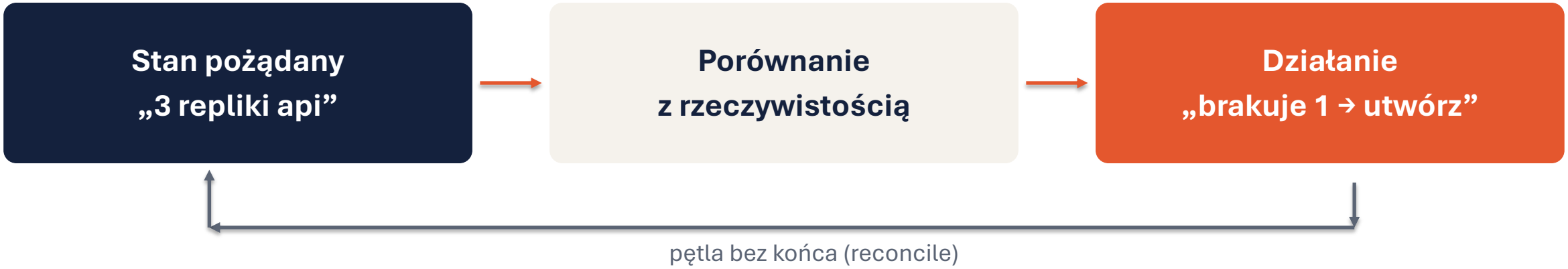
Opisujesz stan, jaki chcesz mieć — Kubernetes pilnuje, żeby tak było

Architektura klastra: control plane i węzły robocze



Wszystko idzie przez API server. Nikt poza nim nie rozmawia z etcd.

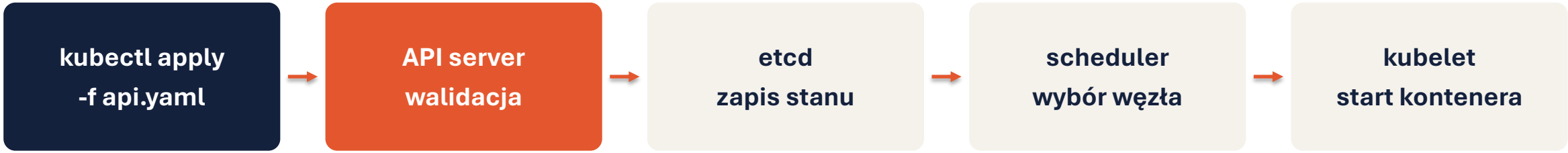
Model deklaratywny: opisujesz stan, a nie kroki



Imperatywnie (jak w skrypcie)	Deklaratywnie (Kubernetes)
• uruchom 3 kontenery • jeśli któryś padnie — Ty go podnosisz • stan wiesz tylko Ty	• plik YAML: replicas: 3 • kubectl apply -f api.yaml • kontroler pilnuje 3 replik zawsze

Pliki YAML w Git = pełny opis środowiska. Na tym stoi GitOps.

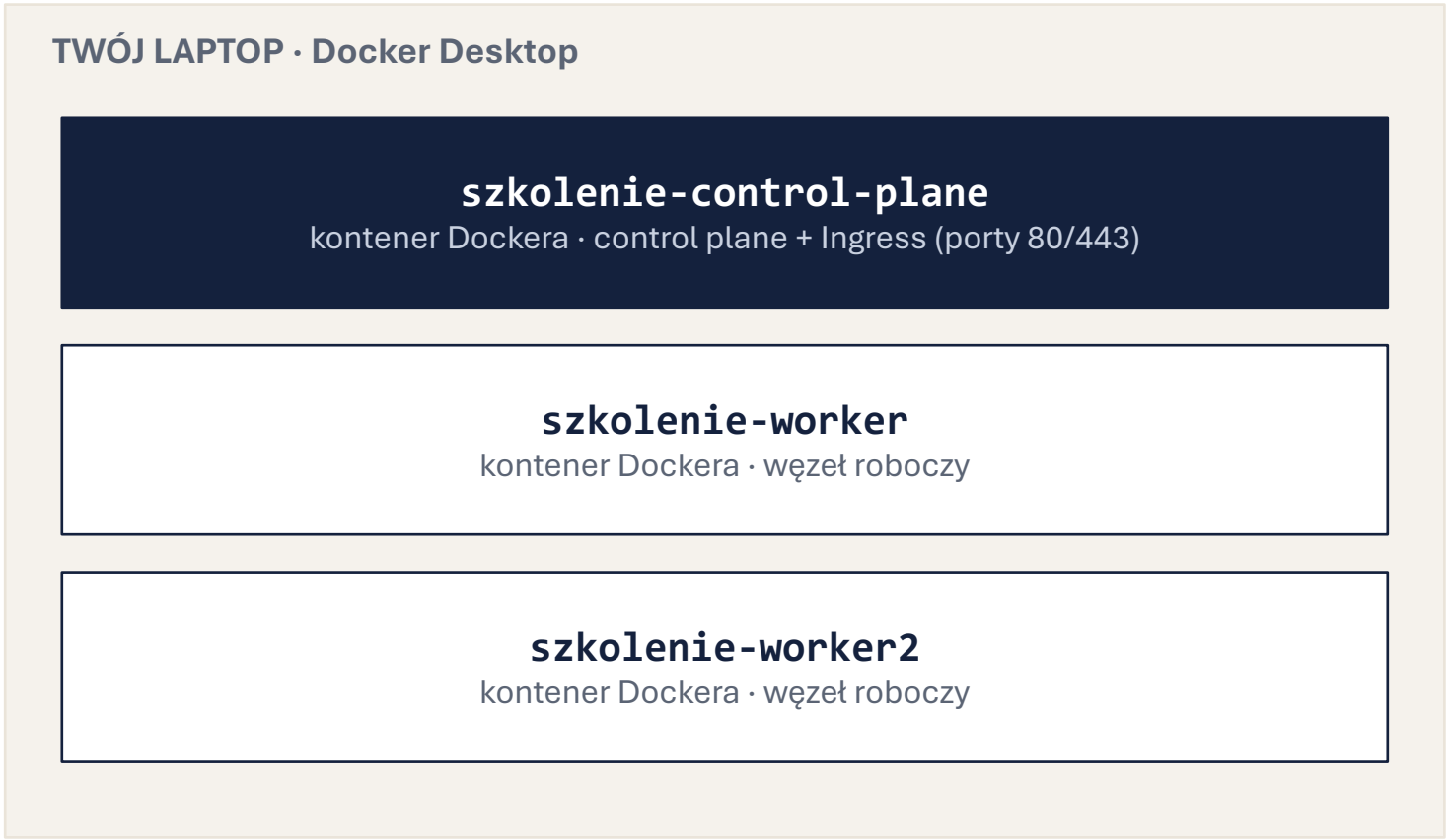
kubectl: jedno narzędzie do całego klastra




```
kubectl get nodes
kubectl get pods -A
kubectl describe pod <nazwa>
kubectl logs <nazwa>
kubectl apply -f plik.yaml
kubectl explain pod.spec
```

- kontekst: do którego klastra mówisz (~/.kube/config)
- namespace: w której „przestrzeni” klastra
- get · describe · logs · apply — 90% pracy
- explain = dokumentacja w terminalu

Nasz klaster: kind — Kubernetes w kontenerach Dockera




```
./setup/start-klastra.sh
kubectl get nodes
```

- prawdziwy Kubernetes, ten sam co w chmurze
- każdy węzeł to kontener — docker ps go pokaże
- start w 2–3 minuty, zero kosztów
- do produkcji: AKS, EKS, GKE, OpenShift...

LAB `labs/lab05-kubectl/`

Twój klaster i kubectl

1. ./setup/start-klastra.sh — trzy węzły Ready
2. kontekst, namespace, komponenty control plane w kube-system
3. kubectl explain i api-resources — dokumentacja w terminalu
4. pierwszy pod: kubectl run, logs, describe, -o yaml

SUKCES: kubectl get nodes pokazuje 3 węzły Ready, a pod ping wypisuje datę co 5 s

Dzień 1: co już umiesz

Docker	Compose	Kubernetes
• uruchamiasz i diagnozujesz kontenery • łączysz je w sieci i trzymasz dane w wolumenach • budujesz mały, bezpieczny obraz	• opisujesz aplikację z kilku usług w jednym pliku • pilnujesz kolejności startu • wiesz, gdzie Compose się kończy	• znasz części klastra i ich role • rozumiesz model deklaratywny • masz własny klaster i kubectl

PYTANIE DO GRUPY

Pod padł o 3 w nocy.
Kto go podniesie w Compose, a kto w Kubernetesie?

Dalej: jutro: Pod, Deployment, Service, Ingress — i ToDos na klastrze